Assignment-2

Build a Decentralized Crowdfunding DApp using Hardhat and React

Total Marks: 110 **Submission Deadline: 18/3/2026**

Objective

Create a decentralized crowdfunding platform where verified users can launch fundraising campaigns and others can contribute using Ether. The DApp will use two smart contracts — KYCRegistry and Crowdfunding — and connect to MetaMask via a React frontend.

Instructions

You are required to develop a complete DApp for crowdfunding using Hardhat (for smart contract development and deployment) and React.js (for frontend integration).

Use the following smart contracts:

- KYCRegistry.sol — handles user verification.
- Crowdfunding.sol — manages campaigns, contributions, and withdrawals.

DApp Requirements

Your DApp should include the following key features:

1. KYC System

- Users can submit a KYC request (name + CNIC).
- The admin can view, approve, or reject KYC requests.
- Only verified users (or admin) can create crowdfunding campaigns.

2. Campaign Creation

- Verified users should be able to create new campaigns by entering:
    - Campaign Title
    - Description
    - Funding Goal (in ETH)
- On submission, the contract should emit a CampaignCreated event.

3. Campaign Listing

- Display all campaigns with details:
    - Title, Description, Goal, Funds Raised, Creator, Status (Active/Completed)
- Each campaign card should have a “Contribute” button.

4. Contributions

- Any user even if His/Her KYC is not approved, can contribute ETH to an active campaign.
- The total funds should update in real time after contribution.
- Automatically mark a campaign as completed when the goal is reached.

5. Withdraw Funds

- The creator of a completed campaign should be able to withdraw funds to their accounts.
- Once withdrawn, the campaign status should update to Withdrawn.

6. MetaMask Integration

- Connect the React frontend to MetaMask wallet.
- Display the user’s wallet address and balance.
- Ensure all contract interactions (create campaign, contribute, withdraw, approve KYC, etc.) are done via MetaMask.

Technical Requirements

- Use Hardhat or any Blockchain for compiling, deploying, and verifying contracts locally.
- Use Ethers.js in your React app for blockchain interactions.
- Display transaction confirmations and error messages in the UI.
- Maintain a clean, user-friendly interface with basic styling.

Deliverables

- Hardhat project folder (with contracts, scripts, and deployments).
- React frontend code folder (with connection to contracts).
- Screenshots with brief notes:
    1. KYC approval Snapshots
    2. Campaign Creating Snapshots

3. Contributions Snapshots
4. Fund withdrawal Snapshots etc.

Anti-Plagiarism Strategy:

Contract Requirements

- The contract names should have your name e.g. ContractName_YourName
- In KYC Approval, Approve your Full name

Frontend Requirements

- The DApp's homepage should display the student's name and roll number e.g. Developed by Your_Name, Roll_No.

Marks Distribution

Component	Marks
1. Smart Contract Implementation	20 marks
2. KYC Verification Flow	15 marks
3. Campaign Management	15 marks
4. Contributions & Withdrawals	15 marks
5. Frontend (React) Functionality	15 marks
6. MetaMask Integration	10 marks
7. Report	20 marks